

期末试题答案汇总

目录

1	编译原理期末试题答案汇总
---	--------------

2

1 编译原理期末试题答案汇总

1.1 使用结构

1. 先看“题型速查”，按同类题合并复习。
2. 再看“通用答题模板”，掌握自动机、LL、LR、回填、SSA、调度、符号执行、指针分析。
3. 最后按年份看题目对应答案。

1.2 题型速查

- 正则表达式、Thompson NFA、子集构造 DFA、DFA 最小化：2021、2022、2023、2025。复习重点是逐层画 NFA，DFA 状态写成 NFA 状态集合，最小化按分组细化。
- LL(1)：2021、2022、2023、2025。复习重点是消除左递归、FIRST、FOLLOW、预测分析表、冲突判断。
- LR/SLR/LR(1)：2021、2022、2023。复习重点是 closure、goto、项目集、归约条件、冲突说明。
- ANTLR4 优先级上升：2022、2023。复习重点是运算符优先级、结合性、改写文法、析树。
- 语法制导定义与翻译：2022、2023。复习重点是属性定义、综合/继承属性、语义规则。
- 中间代码与回填：2022、2023。复习重点是 truelist、falselist、nextlist、while/if 回填。
- SSA 与支配关系：2025。复习重点是 Dom、Post-Dom、Dom Frontier、支配树。
- 指令调度：2025。复习重点是 True/Anti/Output 依赖、依赖图、重命名、拓扑调度。
- 符号执行与 SAT：2025。复习重点是敏感性、CNF、Tseitin、同时可满足。
- 指针别名：2025。复习重点是 Andersen 约束、图闭包、points-to 集合。

1.3 通用答题模板

1.3.1 模板 1：Thompson 构造

```
epsilon: start --epsilon--> end
a      : start --a--> end
A|B    : 新 start epsilon 到 A.start/B.start, A.end/B.end epsilon 到新 end
AB     : A.end epsilon 到 B.start
A*     : 新 start epsilon 到 A.start 和新 end; A.end epsilon 到 A.start 和新 end
```

1.3.2 模板 2: 子集构造

```

Start = epsilon-closure({NFA start})
对每个 DFA 状态 S 和字符 a:
    T = epsilon-closure(move(S, a))
    若 T 新出现, 加入状态集合
终止状态:  $S \cap F_{NFA} \neq \text{emptyset}$ 

```

1.3.3 模板 3: DFA 最小化

```

PO = {F, Q-F}
按每个输入字符的转移目标所在组拆分
重复直到分组不再变化
每个分组合并成一个状态

```

1.3.4 模板 4: LL(1)

1. 消除左递归
2. 计算 FIRST
3. 计算 FOLLOW
4. 对 $A \rightarrow \alpha$:
FIRST(α)-{epsilon} 中每个 a 填 M[A,a]
若 epsilon in FIRST(α), FOLLOW(A) 中每个 b 填 M[A,b]
5. 表中无多重入口, 则是 LL(1)

1.3.5 模板 5: LR(1)

```

项目: [A -> alpha . beta, a]
closure:
    若 [A -> alpha . B beta, a] 在 I 中,
    对 B -> gamma 和 b in FIRST(beta a), 加入 [B -> . gamma, b]
goto(I,X):
    closure({[A -> alpha X . beta, a]})
归约:
    [A -> alpha ., a] 只在 a 列填 reduce

```

1.3.6 模板 6: 回填

```

B.truelist, B.falselist, S.nextlist
makelist(i), merge(p,q), backpatch(p,L)
while:
    记录循环入口 M1
    计算 B
    回填 B.truelist 到循环体入口
    回填循环体 nextlist 到 M1
    发出 goto M1
S.nextlist = B.falselist

```

1.3.7 模板 7：支配与支配边界

$\text{Dom}(A)$: 从入口到 A 的所有路径都经过的块

$\text{PostDom}(A)$: 从 A 到出口的所有路径都经过的块

$\text{idom}(A)$: A 最近的严格支配者

$\text{DF}(B)$: B 支配某个前驱, 但不严格支配该节点的位置

1.3.8 模板 8: Andersen 约束

```

y = &x : x in pts(y)
y = x   : pts(x) subset pts(y)
*y = x  : for v in pts(y), pts(x) subset pts(v)
y = *x  : for v in pts(x), pts(v) subset pts(y)

```

1.4 2025 答案

1.4.1 一、词法分析：正则表达式到 NFA、DFA、最小化

题型：把题面正则表达式按 Thompson 构造为 NFA，再做子集构造和 DFA 最小化。

答案步骤：

1. 按括号最内层到最外层拆正则表达式。
2. 对 ϵ 、字符、连接、选择、闭包分别套 Thompson 模板。
3. 标号 NFA 状态，写出 ϵ -closure。
4. 从起始集合开始列 DFA 转移表。
5. 含 NFA 终止状态的集合标为 DFA 终止状态。
6. 按终止/非终止初分组，迭代拆分，得到最小 DFA。

1.4.2 二、语法分析：消除左递归、FIRST/FOLLOW、预测表

题面文法：

```

A -> B C
B -> B P | r
C -> t | u

```

消除 B 的直接左递归：

```

A  -> B C
B  -> r B'
B' -> P B' | epsilon
C  -> t | u

```

FIRST:

```

FIRST(B)  = {r}
FIRST(B') = {P, epsilon}
FIRST(C)  = {t, u}
FIRST(A)  = {r}

```

FOLLOW:

```

FOLLOW(A) = {$}
FOLLOW(B) = {t, u}
FOLLOW(B') = {t, u}
FOLLOW(C) = {$}

```

预测表:

```

M[A, r] = A -> B C
M[B, r] = B -> r B'
M[B', P] = B' -> P B'
M[B', t] = B' -> epsilon
M[B', u] = B' -> epsilon
M[C, t] = C -> t
M[C, u] = C -> u

```

结论: 表中无多重入口, 消除左递归后的文法是 LL(1)。

1.4.3 三、指令调度

目的:

- 让独立指令靠近, 减少流水线停顿。
- 提高硬件并行执行利用率。

依赖定义:

```

True dependence / RAW : i 写 x, j 读 x, i 必须在 j 前。
Anti dependence / WAR : i 读 x, j 写 x, j 提前会破坏 i 读到的值。
Output dependence / WAW: i 写 x, j 写 x, 顺序影响最终写入值。

```

构建依赖图：

1. 每条指令一个节点。
2. RAW 画实线边。
3. WAR 和 WAW 画虚线边。
4. 寄存器重命名可消除 WAR 和 WAW。
5. 对保留依赖图做拓扑排序，得到调度序列。

1.4.4 四、支配问题

定义：

Dom(n)：从入口到 **n** 的每条路径都经过的节点集合。

PostDom(n)：从 **n** 到出口的每条路径都经过的节点集合。

Dom Frontier(n)：**n** 支配某个前驱，但不严格支配该节点的节点集合。

支配关系图是森林的证明：

1. 除入口节点外，每个节点有唯一直接支配者。
2. 直接支配者严格支配该节点。
3. 严格支配关系无环。
4. 因此每个节点至多一个父节点，且无环，构成以入口为根的树；多入口情形构成森林。

1.4.5 五、符号执行

敏感性定义：

Path-sensitive：区分不同执行路径。

Flow-sensitive：区分程序点和语句顺序。

Context-sensitive：区分函数调用上下文。

CNF：

若干子句的合取，每个子句是文字的析取。

(l11 or l12) and (l21 or l22 or l23)

Tseitin 变换：

1. 为每个子表达式建新变量。
2. 加入新变量与子表达式等价的 CNF 约束。
3. 加入根变量为真。
4. 得到与原式同时可满足的 CNF。

题中同时可满足证明模板：

- 对辅助变量 c 分情况。
- 若左式可满足，则赋值满足其中一个含 c 子句和另一个含 $\text{not } c$ 子句，去掉 c 后得到大析取式可满足。
- 若大析取式可满足，则若某个 a_i 为真，令 $c=\text{false}$ ；若某个 b_j 为真，令 $c=\text{true}$ ，左式可满足。
- 两边可满足性等价。

1.4.6 六、指针别名：Andersen

题面语句模板：

```
p = &a;
q = &b;
*p = q;
r = &c;
s = p;
t = *p;
*s = r;
```

约束：

```
a in pts(p)
b in pts(q)
for v in pts(p), pts(q) subset pts(v)
c in pts(r)
pts(p) subset pts(s)
for v in pts(p), pts(v) subset pts(t)
for v in pts(s), pts(r) subset pts(v)
```

闭包推导：

```

pts(p) = {a}
pts(q) = {b}
由 *p=q 得 pts(a) 包含 {b}
pts(r) = {c}
由 s=p 得 pts(s) 包含 {a}
由 t=*p 且 pts(p)={a} 得 pts(a) subset pts(t), 所以 pts(t) 包含 {b}
由 *s=r 且 pts(s)={a} 得 pts(a) 包含 {c}
最终 pts(a) = {b,c}, pts(t) 包含 {b,c}

```

1.5 2023 答案

1.5.1 题 1: 正则表达式与自动机

题型同 2025 第一题。答案按 Thompson、子集构造、DFA 最小化三步写。

1.5.2 题 2: LL 语法分析

文法:

```

S -> cS | AB
A -> aA | epsilon
B -> bBa | d

```

FIRST:

```

FIRST(A) = {a, epsilon}
FIRST(B) = {b, d}
FIRST(S) = {c, a, b, d}

```

FOLLOW:

```

FOLLOW(S) = {$}
FOLLOW(A) = FIRST(B) = {b, d}
FOLLOW(B) = FOLLOW(S) = {$}

```

预测表:

```

M[S,c] = S -> cS
M[S,a] = S -> AB
M[S,b] = S -> AB
M[S,d] = S -> AB
M[A,a] = A -> aA
M[A,b] = A -> epsilon
M[A,d] = A -> epsilon
M[B,b] = B -> bBa
M[B,d] = B -> d

```

结论: 无冲突, 是 LL(1) 文法。

1.5.3 题 3: ANTLR4 优先级上升

原文法:

```
S -> E
E -> E ! | E ^ E | E + E | INT
```

优先级:

```
! 最高, 后缀
^ 右结合
+ 最低, 左结合
```

答题要点:

- 先把 $5!!$ 归为后缀链。
- 2^3^4 按右结合归为 $2^{(3^4)}$ 。
- 最外层用 $+$ 左结合: $(1 + (2^{(3^4)})) + ((5!)!)$ 。

1.5.4 题 4: 语法制导翻译

任务: 二进制小数转十进制。

可定义属性:

```
S.val
L.val, L.len
B.val
```

规则:

```
S -> L1 . L2    S.val = L1.val + L2.val / 2^(L2.len)
S -> L           S.val = L.val
L -> L1 B        L.val = L1.val * 2 + B.val; L.len = L1.len + 1
L -> B           L.val = B.val; L.len = 1
B -> 0           B.val = 0
B -> 1           B.val = 1
```

1.5.5 题 5: 中间代码生成

二分查找 while 部分按回填模板写。关键结构:

```

Lbegin:
if low <= high goto Lbody
goto Lexit
Lbody:
mid = (low + high) / 2
if dict[mid] < key goto Lless
goto Lnotless
Lless:
low = mid + 1
goto Lbegin
Lnotless:
if dict[mid] > key goto Lgreater
goto Lequal
Lgreater:
high = mid - 1
goto Lbegin
Lequal:
return mid
Lexit:
return -1

```

1.6 2022 答案

1.6.1 题 1: 正则表达式与自动机

表达式: $(a \mid b)^* a^+$ 。

作答:

- 对 $(a|b)$ 构造选择 NFA。
- 对 $(a|b)^*$ 加星闭包。
- 对 a^+ 写成 $a a^*$ 。
- 将两部分连接。
- 子集构造时 DFA 起始状态为起始点 epsilon-closure。
- 终止状态为包含 NFA 终点的集合。

1.6.2 题 2: LL 语法分析

文法:

```

D -> T L
T -> int | real
L -> L , id | id

```

消除 L 左递归:

```
D -> T L
T -> int | real
L -> id L'
L' -> , id L' | epsilon
```

FIRST:

```
FIRST(D) = {int, real}
FIRST(T) = {int, real}
FIRST(L) = {id}
FIRST(L') = {' ', epsilon}
```

FOLLOW:

```
FOLLOW(D) = {$}
FOLLOW(T) = {id}
FOLLOW(L) = {$}
FOLLOW(L') = {$}
```

预测表:

```
M[D,int] = D -> T L
M[D,real] = D -> T L
M[T,int] = T -> int
M[T,real] = T -> real
M[L,id] = L -> id L'
M[L',,] = L' -> , id L'
M[L', $] = L' -> epsilon
```

结论: 无冲突, 是 LL(1)。

1.6.3 题 5: 语法制导定义

任务 1: 统计声明个数。

```
P -> D          P.count = D.count
D -> D1 ; D2     D.count = D1.count + D2.count
D -> T id        D.count = 1
D -> func id {D1} D.count = 1 + D1.count
```

任务 2: 打印变量嵌套深度。

```
P -> D          D.depth = 0
D -> D1 ; D2     D1.depth = D.depth; D2.depth = D.depth
D -> T id        print(id, D.depth)
D -> func id {D1} D1.depth = D.depth + 1
```

1.6.4 题 6：中间代码生成

GCD while 部分：

```
Lbegin:
if a != b goto Lbody
goto Lexit
Lbody:
if a > b goto Lthen
goto Lelse
Lthen:
a = a - b
goto Lbegin
Lelse:
b = b - a
goto Lbegin
Lexit:
return a
```

1.7 2021 与 2020 答案

PDF 文本抽取保留了题型主干，答案按通用模板作答：

- 正则表达式与自动机：Thompson NFA、子集构造 DFA、DFA 最小化。
- LL：消除左递归、FIRST、FOLLOW、预测表、LL(1) 判断。
- LR：FIRST/FOLLOW、LR(1) 自动机、LR(1) 分析表、冲突判断。
- CFG/CNF：按 Chomsky Normal Form 要求改写产生式。
- 语法制导翻译：定义综合属性和继承属性，写产生式对应语义规则。
- 目标代码生成：按题面给定指令集合填空，遵守调用约定、条件跳转、栈帧和返回值规则。